Guion de trabajo tutorizado

Caso Retail

Business Analytics

Profesor José Ignacio Merino
Máster Big Data & Business Analytics (MBDA)

Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «Trabajo Tutorizado - Caso Retail.pdf» (Campus Virtual UPO).

Objetivos

• Realización de un Forecast con Python

Actividades

A1. (2.5 puntos) Considere el dataset day.csv que contiene los alquileres diarios de bicicletas en una ciudad que encontrará en https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset. Tiene que tener en cuenta que la columna "casual" son las bicis alquiladas a usuarios noregistrados, "registered" son las alquiladas a usuarios registrados y "cnt" son las alquiladas en total (por lo que casual + registered = cnt). Es decir, elimine las variables casual y registered si su objetivo es predecir cnt, pues de lo contrario estará introduciendo la solución al modelo y por tanto le dará un error de 0.

Tiene que generar un modelo para cnt usando LinearModel, RandomForestRegressor y GradientBoostingRegressor encontrando los mejores parámetros y discutiendo cuál de los 3 modelos da un mejor resultado sobre el mismo conjunto de train/test.

A2. (2.5 puntos) Considere las ventas del fichero que se le proporciona en el fichero "Ventas mensuales de Fanta 33cl.csv". En el podrá encontrar las ventas mensuales de Fanta de 33cl desde Enero de 2013 a. ¿Podría hacer un forecast con ARIMA de los próximos 12 meses sobre las unidades de Fanta que vamos a vender?

b. En caso tuviésemos en el almacén 1000 latas de fanta, ¿podría decir para cuántos días de stock hay provisiones?

A3. (2.5 puntos) Considere ahora el fichero de "Ventas diarias de Fanta 33cl.csv". En él podrá encontrar las ventas diarias de Fanta de 33 cl en un establecimiento junto con las siguientes columnas:

1. Fecha
2. Unidades: Lata vendidas ese día
3. Precio: El precio al que se ha vendido ese día la lata de Fanta
4. Mes: La parte del mes de la fecha
5. Anno: La parte del año de la fecha
6. DiaSemana: El día de la semana (1->Domingo, 2-> Lunes, 3-> Martes, …)
7. T: La temperatura en ºC de ese día

Haga un forecast usando LinearRegression, RandomForestRegressor y GradientBoostingRegressor tomando una muestra de train/test igual para los tres modelos. Obtenga cuál de ellos da un mejor resultado. Una vez hecho, conteste a las siguientes cuestiones:

a. ¿Qué problemas ve a aplicar este modelo para predecir?

b. ¿Qué ventajas tiene sobre otros métodos como ARIMA?

c. ¿Qué ventajas ve que ofrece ARIMA sobre este método?

A4. (2.5) Se proporciona el consumo de agua de la Ciudad de Sevilla considerando datos mensuales y diarios. En los datos diarios, se adjunta el día de la semana, mes, año, temperatura y precipitación de ese día. Obtenga:

- Un modelo de forecast usando datos diarios
- Un modelo de forecast usando datos mensuales

En el dataset de los datos diarios, hay varios NA’s en el campo de las temperaturas debido a que algunos días la estación meteorológica falló y no registró datos. Posiblemente obtenga algún error debido a esto, así pues, para poder quitarlos use la función .dropna() en el dataset. Por ejemplo, si quisiese quitar todos los NAs de un dataset llamado agua y quedarme únicamente con aquellas filas que tuviesen todos los datos el comando sería:

aguaSinNA = agua.dropna()

a. Haga un train/test usando los 6 últimos meses en el caso de la predicción mensual y de 3 meses en el caso de la predicción diaria y entregue la comparación entre el forecast y el valor real de forma gráfica en el conjunto de test b. Con una de las dos métricas (RSME ó MAE), obtenga la fiabilidad del forecast

Fecha y Modo de entrega

Antes del día 29 de Junio de 2026

RESOLUCIÓN

Entorno: Python 3.11 con pandas, scikit-learn 1.8 y statsmodels. Los scripts completos se incluyen a continuación y también en la carpeta de la actividad. Los datos son los CSV suministrados en el Aula Virtual y `day.csv` del *Bike Sharing Dataset* (UCI, 731 días de 2011-2012).

A1. Alquiler diario de bicicletas (cnt)

Se eliminan `instant` y `dteday` (identificador y fecha) y, sobre todo, `casual` y `registered`. Como `casual + registered = cnt`, dejarlas daría un error prácticamente nulo por **fuga de información**. Las variables categóricas (estación, meteorología, mes y día de la semana) se codifican con *one-hot* y se usa el **mismo split 80/20** (`random_state=42`) para todos los modelos. Los hiperparámetros se buscan con `GridSearchCV` (validación cruzada de 5 particiones sobre train, optimizando MAE) y se evalúa una única vez sobre test.

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
df = pd.read_csv('day.csv')
X = df.drop(columns=['instant','dteday','casual','registered','cnt'])   # casual+registered
= cnt -> fuga de información
X = pd.get_dummies(X, columns=['season','weathersit','mnth','weekday'], drop_first=True)
y = df['cnt']
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)
print('train', Xtr.shape, 'test', Xte.shape)
modelos = {
 'LinearModel (Ridge)': (Ridge(), {'alpha':[0.01,0.1,1,10]}),
 'RandomForestRegressor': (RandomForestRegressor(random_state=42), {'n_estimators':
[200,500],'max_depth':[None,10,20],'min_samples_leaf':[1,3]}),
 'GradientBoostingRegressor': (GradientBoostingRegressor(random_state=42), {'n_estimators':
[200,500],'learning_rate':[0.03,0.1],'max_depth':[2,3,4]}),
}
res=[]
lr = LinearRegression().fit(Xtr,ytr); p=lr.predict(Xte)
res.append(['LinearRegression (sin
hiperparámetros)','-',mean_absolute_error(yte,p),mean_squared_error(yte,p)**.5,r2_score(yte
,p)])
for n,(m,g) in modelos.items():
    gs = GridSearchCV(m, g, cv=5, scoring='neg_mean_absolute_error', n_jobs=-
1).fit(Xtr,ytr)
    p = gs.predict(Xte)
    res.append([n, gs.best_params_, mean_absolute_error(yte,p),
mean_squared_error(yte,p)**.5, r2_score(yte,p)])
r = pd.DataFrame(res, columns=['modelo','mejores_parametros','MAE','RMSE','R2'])
pd.set_option('display.width',200); pd.set_option('display.max_colwidth',80)
print(r.round(3).to_string(index=False))
print('Mejor modelo (MAE):', r.loc[r.MAE.idxmin(),'modelo'])
```

```
aramber@localhost:~$ python3 a1_bicis.py
train (584, 29) test (147, 29)
                          modelo                                                mejores_parametros     MAE    RMSE    R2
LinearRegression (sin hiperparámetros)                                                          - 583.020 796.462 0.842
                LinearModel (Ridge)                                                  {'alpha': 0.1} 582.053 795.774 0.842
              RandomForestRegressor {'max_depth': None, 'min_samples_leaf': 1, 'n_estimators': 500} 470.458 714.226 0.873
          GradientBoostingRegressor     {'learning_rate': 0.1, 'max_depth': 2, 'n_estimators': 200} 475.195 674.183 0.887
Mejor modelo (MAE): RandomForestRegressor
```

Figura 1. Resultado real de A1: mejores hiperparámetros y métricas en test de los modelos lineal, Random Forest y Gradient Boosting.

Modelo	Mejores parámetros	MAE	RMSE	R^2
LinearRegression / Ridge	alpha = 0.1	582	796	0.842
RandomForestRegressor	500 árboles, max_depth=None, min_samples_leaf=1	470	714	0.873
GradientBoostingRegressor	200 árboles, lr=0.1, max_depth=2	475	674	0.887

Discusión. Los dos ensembles de árboles superan claramente al modelo lineal (≈ 110 bicis/día menos de error absoluto), porque capturan relaciones no lineales como el efecto de la temperatura, que satura en los días muy calurosos, y las interacciones año × estación. Entre ellos la diferencia es pequeña: **Random Forest** tiene el menor MAE (470) y **Gradient Boosting** el menor RMSE y el mayor R^2 (674; 0.887), es decir, comete menos errores grandes. Con el criterio MAE fijado en la búsqueda, el mejor es Random Forest; si se penalizan más los errores grandes (RMSE), Gradient Boosting. Ambos explican ~88 % de la varianza de `cnt`.

A2. Forecast ARIMA de las ventas mensuales de Fanta 33cl

La serie tiene 54 meses (enero 2013 – junio 2017) y una estacionalidad anual clara, con picos en verano. Se prueban modelos SARIMA(p,d,q)×$(P,D,Q)_{12}$ y se elige el de menor AIC entre los que convergen con previsiones válidas. El ganador es **SARIMA(0,1,1)×$(0,1,1)_{12}$**, el clásico «modelo de las líneas aéreas»: una diferencia regular y otra estacional con términos MA.

```
import pandas as pd, numpy as np, warnings, logging, itertools
warnings.filterwarnings('ignore')
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
import statsmodels.tsa.statespace.mlemodel as _m
warnings.warn = lambda *a, **k: None
s = pd.read_csv('Ventas Mensuales Fanta 33cl.csv', sep=';')
s['Fecha']=pd.to_datetime(s.Fecha, dayfirst=True); s=s.set_index('Fecha')
['Unidades'].asfreq('MS')
print('Serie:', s.index.min().date(), '->', s.index.max().date(), len(s), 'meses')
best=None
for p,d,q in itertools.product(range(3),[0,1],range(3)):
    for P,D,Q in [(0,0,0),(1,0,0),(0,1,0),(1,1,0),(0,1,1)]:
        try:
            m=SARIMAX(s,order=(p,d,q),seasonal_order=(P,D,Q,12)).fit(disp=False,
warn_convergence=False)
            fc_=m.get_forecast(12).predicted_mean
            if not (np.isfinite(m.aic) and np.isfinite(fc_).all() and fc_.min()>0 and
m.aic>100): continue
            if best is None or m.aic<best[0]: best=(m.aic,(p,d,q),(P,D,Q,12),m)
        except Exception: pass
aic,o,so,m=best
print(f'Mejor modelo por AIC: SARIMA{o}x{so}  AIC={aic:.1f}')
fc=m.get_forecast(12); f=fc.predicted_mean.round(0); ci=fc.conf_int().round(0)
out=pd.DataFrame({'forecast':f,'li95':ci.iloc[:,0],'ls95':ci.iloc[:,1]});
out.index=out.index.strftime('%Y-%m')
print(out.to_string())
print('Total previsto 12 meses:', int(f.sum()))
# cobertura de 1000 latas
dia = f.iloc[0]/f.index[0].days_in_month
print(f'Primer mes previsto ({f.index[0]:%Y-%m}): {f.iloc[0]:.0f} uds -> {dia:.1f}
```

```
latas/día')
stock=1000; dias=0; restante=stock
for fecha,v in f.items():
    diario=v/fecha.days_in_month
    if restante>=v: restante-=v; dias+=fecha.days_in_month
    else: dias+=restante/diario; break
print(f'1000 latas cubren aprox. {dias:.1f} días de venta')
ax=s.plot(figsize=(10,4),label='histórico'); f.plot(ax=ax,label='forecast');
ax.fill_between(ci.index,ci.iloc[:,0],ci.iloc[:,1],alpha=.2)
ax.legend(); ax.set_title('Fanta 33cl - forecast SARIMA 12 meses'); plt.tight_layout();
plt.savefig('fig/a2_arima.png',dpi=110)
```

```
aramber@localhost:~$ python3 a2_fanta_arima.py
Serie: 2013-01-01 -> 2017-06-01 54 meses
Mejor modelo por AIC: SARIMA(0, 1, 1)x(0, 1, 1, 12)  AIC=550.2
         forecast    li95    ls95
2017-07    1599.0  1270.0  1928.0
2017-08    1574.0  1244.0  1903.0
2017-09    1025.0   695.0  1356.0
2017-10    1035.0   704.0  1365.0
2017-11     742.0   410.0  1073.0
2017-12    1225.0   892.0  1557.0
2018-01     785.0   453.0  1118.0
2018-02     716.0   383.0  1049.0
2018-03     952.0   618.0  1286.0
2018-04    1017.0   683.0  1351.0
2018-05    1108.0   773.0  1443.0
2018-06    1123.0   787.0  1459.0
Total previsto 12 meses: 12901
Primer mes previsto (2017-07): 1599 uds -> 51.6 latas/día
1000 latas cubren aprox. 19.4 días de venta
```

Figura 2. Resultado real de A2: modelo seleccionado, forecast de los próximos 12 meses con intervalo de confianza al 95 % y cobertura de 1000 latas.

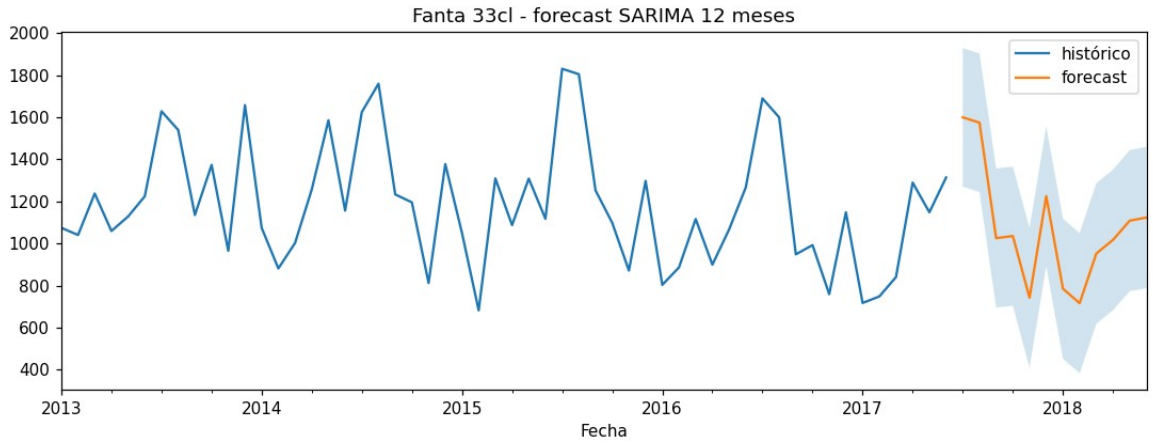


Figura 3. Serie histórica y forecast SARIMA de 12 meses (banda: IC 95 %).

a. Sí: el forecast para julio 2017 – junio 2018 suma **12 901 unidades**, con máximos en julio (1 599) y agosto (1 574) y mínimos en invierno (716 en febrero 2018). Los intervalos de confianza (±330 uds/mes aprox.) reflejan la incertidumbre de una serie corta.

b. Con 1 000 latas en almacén y la demanda prevista para julio (1 599 uds ≈ **51,6 latas/día**), el stock cubre aproximadamente **19 días**. Si se toma como referencia la demanda media anual prevista (12 901/365 ≈ 35 latas/día), la cobertura sería de unos 28 días. Por ello la cobertura depende de la estacionalidad y conviene calcularla con la previsión del mes en curso.

A3. Forecast de ventas diarias de Fanta con modelos de regresión

Variables explicativas: precio, mes, año, día de la semana (*one-hot*) y temperatura. Se usa el mismo split 80/20 (`random_state=7`) para los tres modelos.

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
d = pd.read_csv('Ventas diarias de Fanta 33cl.csv', sep=';', decimal=',')
d['Fecha']=pd.to_datetime(d.Fecha, format='%d/%m/%y')
print(d.describe().round(2).to_string())
X = pd.get_dummies(d[['Precio','Mes','Anno','DiaSemana','T']], columns=['DiaSemana'],
drop_first=True); y=d.Unidades
Xtr,Xte,ytr,yte = train_test_split(X,y,test_size=0.2,random_state=7)
ms={'LinearRegression':LinearRegression(),'RandomForestRegressor':RandomForestRegressor(n_e
stimators=400,min_samples_leaf=3,random_state=7),

'GradientBoostingRegressor':GradientBoostingRegressor(n_estimators=300,learning_rate=0.05,m
ax_depth=3,random_state=7)}
rows=[]
for n,m in ms.items():
    p=m.fit(Xtr,ytr).predict(Xte);
rows.append([n,mean_absolute_error(yte,p),mean_squared_error(yte,p)**.5,r2_score(yte,p)])
r=pd.DataFrame(rows,columns=['modelo','MAE','RMSE','R2']);
print(r.round(3).to_string(index=False))
print('Mejor:', r.loc[r.MAE.idxmin(),'modelo'])
lr=ms['LinearRegression']; print('Coeficientes LR:', {k: round(float(v),2) for k,v in
zip(X.columns, lr.coef_)})
imp=pd.Series(ms['RandomForestRegressor'].feature_importances_,X.columns).sort_values(ascen
ding=False).round(3); print('Importancias RF:', imp.head(6).to_dict())
```

```
aramber@localhost:~$ python3 a3_fanta_diaria.py
                       Fecha  Unidades   Precio      Mes     Anno  DiaSemana        T
count                   1716   1716.00  1716.00  1716.00  1716.00    1716.00  1716.00
mean   2014-11-08 22:57:03.776223     44.08     0.42     6.58  2014.35       4.09    26.22
min           2012-05-02 00:00:00      1.00     0.17     1.00  2012.00       1.00     7.10
25%           2013-09-06 18:00:00     22.00     0.42     4.00  2013.00       2.00    18.20
50%           2014-11-13 12:00:00     38.00     0.43     7.00  2014.00       4.00    25.90
75%           2016-01-22 06:00:00     59.00     0.43    10.00  2016.00       6.00    34.00
max           2017-03-31 00:00:00    207.00     0.59    12.00  2017.00       7.00    45.50
std                           NaN     30.51     0.03     3.45     1.44       1.96     8.82
                  modelo    MAE   RMSE    R2
        LinearRegression 15.237 19.688 0.523
    RandomForestRegressor 14.515 19.048 0.553
GradientBoostingRegressor 14.368 18.796 0.565
Mejor: GradientBoostingRegressor
Coeficientes LR: {'Precio': -150.04, 'Mes': 0.32, 'Anno': -5.08, 'T': 0.81, 'DiaSemana_2': 18.67, 'DiaSemana_3': 21.46, 'DiaSemana_4': 22.44, 'DiaSemana_5': 20.
22, 'DiaSemana_6': 39.77, 'DiaSemana_7': 54.93}
Importancias RF: {'T': 0.257, 'Anno': 0.251, 'DiaSemana_7': 0.187, 'Mes': 0.091, 'DiaSemana_6': 0.074, 'Precio': 0.072}
```

Figura 4. Resultado real de A3: descriptivos, métricas en test, coeficientes del modelo lineal e importancias del Random Forest.

El mejor es **GradientBoostingRegressor** (MAE = 14,4 latas; R^2 = 0,565), seguido de Random Forest (14,5; 0,553) y la regresión lineal (15,2; 0,523). Los coeficientes muestran efectos interpretables: los viernes (+40) y sábados (+55) se venden más que los domingos (categoría base; en la codificación del fichero 1 = domingo y 7 = sábado); cada ºC suma ~0,8 latas; y el precio tiene un efecto fuertemente negativo (−150 por €).

a. Problemas de aplicarlo para predecir:

- Necesita conocer de antemano los regresores futuros, como la **temperatura** de mañana, que es a su vez una previsión con error, y el precio aplicado.

- Ignora la dependencia temporal: no usa ventas pasadas (retardos), tendencia ni autocorrelación. Además, el split aleatorio mezcla días futuros en train, lo que hace optimistas las métricas frente a un *backtest* temporal.
- Los árboles no extrapolan: si el precio o la temperatura salen del rango histórico, el Random Forest o Gradient Boosting devuelve el valor del extremo conocido. `Anno` como variable numérica también extrapola mal.
- $R^2 \approx 0{,}56$: queda mucha variabilidad sin explicar (promociones, festivos, eventos, roturas de stock).

b. Ventajas frente a ARIMA: incorpora variables explicativas (precio, clima, día de la semana), lo que permite análisis *what-if* («¿qué pasa si bajo el precio 5 céntimos?»); capta relaciones no lineales e interacciones; no exige una serie estacionaria ni equiespaciada; y tolera huecos en las fechas.

c. Ventajas de ARIMA: modela explícitamente tendencia, estacionalidad y autocorrelación; solo necesita el propio histórico, sin previsiones de regresores; ofrece **intervalos de confianza** con base estadística; es parsimonioso e interpretable; y está pensado para proyectar hacia el futuro (varios pasos adelante).

A4. Consumo de agua de Sevilla: forecast diario y mensual

Datos diarios (2006-01-01 a 2017-04-04, 4 114 filas). Hay 2 NA en `CONSUMO` y 2 en `TEMPERATURA`; se eliminan con `aguaSinNA = agua.dropna()` (quedan 4 112 filas). El modelo diario es un GradientBoostingRegressor con día de la semana, día del año, tendencia, lluvia, temperatura, mes y el consumo de hace 7 días (`lag7`). **Test: los 3 últimos meses.**

Datos mensuales (enero 2006 – marzo 2017, 135 meses). Se aplica un SARIMA$(1,0,1)\times(0,1,1)_{12}$. **Test: los 6 últimos meses.**

```
import pandas as pd, numpy as np, warnings, logging; warnings.filterwarnings('ignore')
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
from statsmodels.tsa.statespace.sarimax import SARIMAX
import statsmodels.tsa.statespace.mlemodel as _m
warnings.warn = lambda *a, **k: None
a = pd.read_csv('CONSUMO SEVILLA CON MESES.csv', sep=';', decimal=',', encoding='latin-1')
a['Fecha']=pd.to_datetime(a.Timestamp.str.split().str[0], format='%d/%m/%y')
print('Diario:', a.Fecha.min().date(), '->', a.Fecha.max().date(), len(a), 'filas; NAs por
columna:', a.isna().sum().to_dict())
aguaSinNA = a.dropna(); print('Tras dropna():', len(aguaSinNA))
g = aguaSinNA.copy(); g['dow']=g.Fecha.dt.dayofweek; g['doy']=g.Fecha.dt.dayofyear;
g['t']=(g.Fecha-g.Fecha.min()).dt.days
g=g.sort_values('Fecha'); g['lag7']=g.CONSUMO.shift(7); g=g.dropna()
corte = g.Fecha.max()-pd.DateOffset(months=3)
tr,te = g[g.Fecha<=corte], g[g.Fecha>corte]
F=['dow','doy','t','LLUVIA','TEMPERATURA','MES','lag7']
m=GradientBoostingRegressor(n_estimators=400,learning_rate=0.05,max_depth=3,random_state=1)
.fit(tr[F],tr.CONSUMO); p=m.predict(te[F])
mae_d=mean_absolute_error(te.CONSUMO,p); rmse_d=mean_squared_error(te.CONSUMO,p)**.5
print(f'DIARIO  test={te.Fecha.min().date()}..{te.Fecha.max().date()} ({len(te)} días)
MAE={mae_d:.2f}  RMSE={rmse_d:.2f}  MAPE={np.mean(abs(te.CONSUMO-p)/te.CONSUMO)*100:.2f}%')
fig,ax=plt.subplots(figsize=(10,4)); ax.plot(te.Fecha,te.CONSUMO,label='real');
ax.plot(te.Fecha,p,label='forecast GBR'); ax.set_title('Consumo diario de agua Sevilla -
test 3 meses'); ax.legend(); plt.tight_layout(); plt.savefig('fig/a4_diario.png',dpi=110)
mm = pd.read_csv('CONSUMO SEVILLA MENSUAL.csv', sep=';', decimal=',').dropna()
```

```
mes={'ene':1,'feb':2,'mar':3,'abr':4,'may':5,'jun':6,'jul':7,'ago':8,'sep':9,'oct':10,'nov'
:11,'dic':12}
mm['Fecha']=[pd.Timestamp(2000+int(x.split('-')[1]), mes[x.split('-')[0].lower()[:3]], 1)
for x in mm.mes]
s=mm.set_index('Fecha').consumo.asfreq('MS')
print('Mensual:', s.index.min().date(), '->', s.index.max().date(), len(s), 'meses')
tr,te=s.iloc[:-6],s.iloc[-6:]
mod=SARIMAX(tr,order=(1,0,1),seasonal_order=(0,1,1,12)).fit(disp=False,
warn_convergence=False); fc=mod.get_forecast(6).predicted_mean
mae_m=mean_absolute_error(te,fc); rmse_m=mean_squared_error(te,fc)**.5
print(pd.DataFrame({'real':te.round(1),'forecast':fc.round(1),'error%':((fc-te)/
te*100).round(2)}).to_string())
print(f'MENSUAL SARIMA(1,0,1)x(0,1,1,12)  MAE={mae_m:.1f}  RMSE={rmse_m:.1f}
MAPE={np.mean(abs(te-fc)/te)*100:.2f}%')
fig,ax=plt.subplots(figsize=(10,4)); s.iloc[-36:].plot(ax=ax,label='real');
fc.plot(ax=ax,label='forecast (test 6 meses)',marker='o'); ax.legend();
ax.set_title('Consumo mensual de agua Sevilla'); plt.tight_layout();
plt.savefig('fig/a4_mensual.png',dpi=110)
```

```
aramber@localhost:~$ python3 a4_agua.py
Diario: 2006-01-01 -> 2017-04-04 4114 filas; NAs por columna: {'Timestamp': 0, 'DIASEM': 0, 'CONSUMO': 2, 'LLUVIA': 0, 'TEMPERATURA': 2, 'ANNO': 0, 'MES': 0, 'F
echa': 0}
Tras dropna(): 4112
DIARIO  test=2017-01-05..2017-04-04 (90 días)  MAE=5.28  RMSE=6.98  MAPE=2.65%
Mensual: 2006-01-01 -> 2017-03-01 135 meses
            real  forecast  error%
2016-10-01  6695.7    6763.6    1.01
2016-11-01  6261.3    6339.6    1.25
2016-12-01  6267.6    6396.3    2.05
2017-01-01  6321.6    6499.5    2.81
2017-02-01  5605.0    5874.1    4.80
2017-03-01  6340.2    6585.5    3.87
MENSUAL SARIMA(1,0,1)x(0,1,1,12)  MAE=161.2  RMSE=178.7  MAPE=2.63%
```

Figura 5. Resultado real de A4: tratamiento de NA, métricas del modelo diario y comparación real/forecast del modelo mensual.

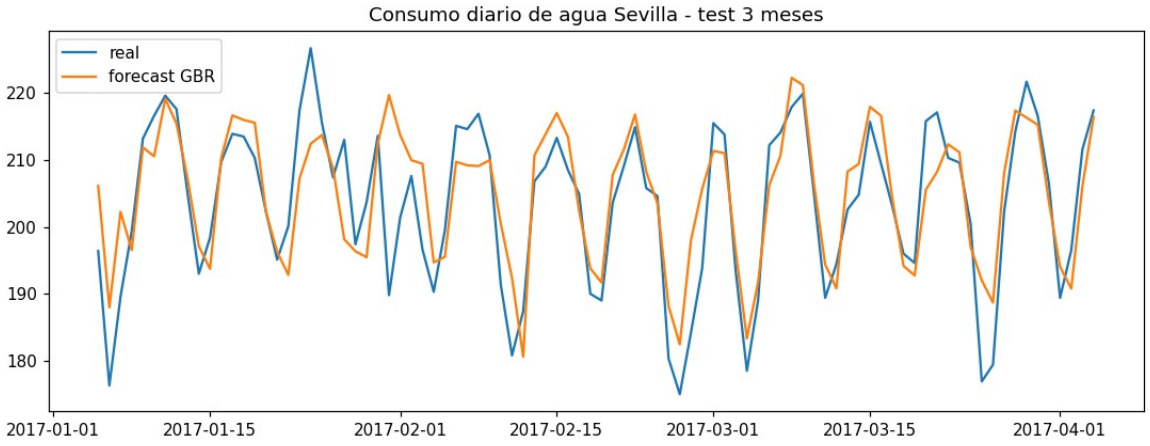


Figura 6. A4 (a). Forecast diario frente a consumo real en el conjunto de test (5 enero – 4 abril 2017).

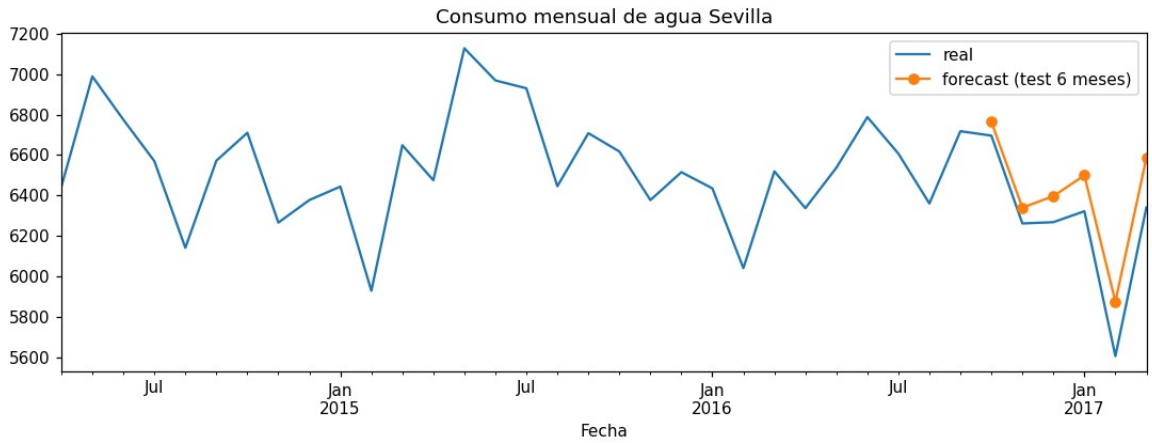


Figura 7. A4 (a). Forecast mensual (6 meses de test, octubre 2016 – marzo 2017) frente a la serie real.

b. Fiabilidad del forecast:

Modelo	Test	MAE	RMSE	MAPE
Diario (GBR)	90 días	5,28	6,98	2,65 %
Mensual (SARIMA)	6 meses	161,2	178,7	2,63 %

Ambos modelos tienen un error relativo muy bajo (≈ 2,6 %). El diario falla en promedio en 5,3 unidades/día sobre un consumo medio de ~200 y sigue bien el patrón semanal. El mensual sobreestima de forma sistemática los últimos meses (+1 % a +4,8 %), lo que apunta a una tendencia descendente del consumo que el modelo no termina de recoger. Podría corregirse con un término de tendencia o reentrenando con los datos más recientes. El MAE mensual (161) no es comparable directamente con el diario porque la escala es ~30 veces mayor.

Referencias

Fanaee-T, H. (2013). *Bike Sharing Dataset*. UCI Machine Learning Repository. https://doi.org/10.24432/C5W894

scikit-learn: GridSearchCV, RandomForestRegressor, GradientBoostingRegressor – https://scikit-learn.org/stable/

statsmodels: SARIMAX – https://www.statsmodels.org/stable/generated/statsmodels.tsa.statespace.sarimax.SARIMAX.html

Hyndman, R.J. & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts.